Leader election on a unidirectional ring: Algorithm B of Dolev, Klawe and Rodeh

Roman Volochai

Distributed Systems, talk of 4 November 2026
Code, slides and this report:
`https://github.com/RomanVolochai/distributed-systems-unidir-leader-finder-b`

This report accompanies the talk on Section 3 of D. Dolev, M. Klawe, M. Rodeh, *An $O(n \log n)$ unidirectional distributed algorithm for extrema finding in a circle*, J. Algorithms 3 (1982), 245–260. It covers the same parts as the talk, fills in what the talk had no time for, and sketches the proofs, keeping the paper's numbering. Section 6 collects the points that are easy to misunderstand. Throughout, log is the binary logarithm.

1 Problem

A ring consists of n processes. Every process sends only to its right neighbour and receives only from its left neighbour. The system is **asynchronous**: delays are arbitrary but finite, every link is **FIFO**, and a process handles its messages one by one. Ids are distinct, all processes run the same program, and n is not known. The task is to find the maximum id; its owner becomes the leader. The cost is the number of messages in the worst case. The lower bound is $\Omega(n \log n)$ (Burns, 1980). Hirschberg and Sinclair conjectured that one-way rings need $\Theta(n^2)$ messages; the simple Algorithm A of the paper (found independently by Peterson) refutes this with $2n \log n + n$, and Algorithm B lowers the constant to $\frac{3}{2}$.

2 Idea: from Algorithm A to Algorithm B

Algorithm A (paper, Section 2). Because links are FIFO, an execution can be seen as a sequence of *phases*. In a phase the active processes form a smaller ring; the passive ones only relay. For three consecutive active processes $u \to v \to w$, v cannot look to the right, so w decides: v's M1 brings $\max(v)$, and v forwards $\max(u)$ in an M2. If $\max(v)$ is larger than both neighbours (a local maximum), w takes it over. At most half of the values are local maxima, so there are at most $\log n + 1$ phases. Every phase but the last puts one M1 and one M2 on every link, the last one only M1s: $2n \log n + n$ messages.

Goal. M1 is needed on every link (each active process must announce its value). Algorithm B keeps it and sends M2 over only about half of the links: $n \log n + \frac{1}{2} n \log n$. The paper gets there with three changes applied to A one after another (paper, Section 3.1).

Change 1: only a winner sends M2. When u's M1 arrives, v already knows whether $\max(u) > \max(v)$. If $\max(u) > \max(v)$, w drops out whatever $\max(w)$ is, so v sends nothing. If instead $\max(v) > \max(u)$, v becomes passive and sends an M2 with *its own* value. The process w compares $\max(v)$ with $\max(w)$ when v's M1 arrives. If $\max(v)$ is bigger, w stores it and enters the new state *waiting*; otherwise w becomes passive and has to drop v's M2 (in B this is a special case of change 2). A waiting process that receives an M2 becomes active in the next phase. If the next M1 comes first, FIFO guarantees that no M2 is coming, and it becomes passive ("two M1s in a row"). The survivors are still the local maxima; the paper remarks that this change alone does not reach $\frac{3}{2} n \log n$.

Change 2: passive processes drop hopeless M2s. A passive process x keeps $\max(x)$, the largest value it has stored: its own id, a value adopted while active and, once change 3 is added, values that passed it with counter ≥ 1. Let u and v be neighbouring active processes, u sending its M2 to v. If such an x lies between them and $\max(x) > \max(u)$, then $\max(u)$ cannot be the maximum, and x drops u's M2. The M1 must still pass, because v needs it to decide about its own value. Here the value carried by the M2 (change 1) pays off: the passive process can compare it with its own max.

Change 3: a counter 2^p. After a few phases the active processes may sit on a short arc; then one M2 still crosses almost the whole ring. The M1 of phase p therefore carries a counter 2^p, decreased by passive and waiting processes. A passive process that receives counter 1 (at distance 2^p) with a value of at least its max becomes waiting, a "virtual candidate"; after it the counter is 0 and the M1 is only relayed. Waiting processes never forward M2, so an M2 of phase p stops within 2^p links. The paper stresses that this change alters the behaviour: passive processes can become active again, and survivors need not be local maxima, so the proof for A no longer applies.

Algorithm B (paper, Section 3.2). State: *phase*, *state*, *id*, max (initially *id*); messages M1 $= (1, i, p, c)$ with value i, phase p and counter c, and M2 $= (2, j)$.

from	to	when	sends
start	active	$phase := 0$, max $:= id$	M1$(id, 0, 1)$
active (B1)	waiting	M1 with $i >$ max; max $:= i$	—
active (B1)	passive	M1 with $i <$ max	M2(max)
waiting (B2)	active	M2 arrives; $phase{+}{+}$	M1$(\max, phase, 2^{phase})$
waiting (B2)	passive	M1 arrives first; then as B3	—
passive (B3)	waiting	M1 with $c = 1$, $i \geq$ max; max $:= i$, $phase := p$	M1 with $c = 0$
passive (B3)	passive	other M1: $i \geq$ max, $c > 1$: max $:= i$, $c{-}{-}$; else $c := 0$	relay
passive (B3)	passive	M2 with $j <$ max: drop; otherwise forward	M2(j) or —
any	leader	M1 with $i = id$	announcement

3 Correctness (Section 3.3 of the paper)

Notation. $A(p)$ is the set of processes that start phase p as active (they send an M1 with phase p); we call them the active processes of phase p. The value of $v \in A(p)$ at that moment is p-max(v) (the p in the name is the phase); origin(j), the *owner* of j, is the process whose id is j; $d(u, v)$ is the number of links from u to v going right. The running example is the ring $5, 2, 6, 3, 1, 4, 8, 7$ from the talk (positions P0–P7), where $A(1) = \{$P3 with 6, P7 with 8$\}$ and $A(2) = \{$P1 with 8$\}$.

Three facts used everywhere.

(F1) *Path of M1.* Passive and waiting processes always forward an M1, possibly with a new counter. Only an active process stops it, and at the very end the owner of its value. So the M1 of $v \in A(p)$ passes every process up to the next process of $A(p)$ and ends there. If $|A(p)| \geq 2$, every process receives exactly one M1 of phase p; this is what makes phases well defined although nothing is synchronised.

(F2) *Order on a link.* An M2 belongs to phase p if a process of $A(p)$ created it in reply to the M1 of phase p it received. On every link all messages of phase p pass before the M1 of phase $p + 1$. Reason: that M1 comes from the nearest $z \in A(p + 1)$ to the left of the link; z sent it only after taking the M2 of its predecessor in $A(p)$, which in turn was sent after that predecessor's M1 of phase p. Every process forwards messages in arrival order and links are FIFO, so by induction along the ring the phase-p messages of the processes in between are also ahead of it. In particular, if a waiting process receives an M1 first, the M2 will never come, which justifies step B2.

(F3) *The counter.* An M1 sent with counter 2^p arrives at distance $d \le 2^p$ with counter $2^p - d + 1$ (and with 0 beyond), unless some process on the way had a larger max and set it to 0. So counter 1 arrives *exactly* at distance 2^p, and only if nobody before had a larger max.

(F1), (F2), the auxiliary fact below and Lemma 3.3.6 depend on each other. Strictly speaking they are proved together by induction on the phase; they are listed separately to keep them readable.

3.3.1 (distinct values). *The values p-max$(\cdot)$ in $A(p)$ are distinct.* Induction on p. A process enters $A(p+1)$ only by taking an M2 of phase p (B2). That M2 was created by a process of $A(p)$, which puts its own value in it and creates at most one M2; by induction these values are distinct. The waiting process that takes an M2 does not forward it, so one M2 activates at most one process.

Where an M2 ends (auxiliary fact). Let $u \in A(p)$ send an M2 with value j, and let w be the next process of $A(p)$. The M2 never passes w: before it, u's M1 reached w (F1, F2), and w then either waits for j or became passive with $\max(w) > j$; in neither case does it forward M2(j). On the way the M2 is either dropped by a passive process with a larger max or taken by a waiting process. The only process between u and w that can be waiting when the M2 reaches it is the one that got counter 1 from u's M1 (every other one got that M1 and stopped waiting by B2), and it waits for j.

3.3.2 (short M2s). *An M2 of phase p travels at most 2^p links.* Let u, j, w be as above. (i) If some process at distance $< 2^p$ from u had max $> j$ when u's M1 passed, it drops the M2: max never decreases, and the process is passive when the M2 arrives (the only waiting process on the way waits for j). (ii) Otherwise, if $d(u, w) \le 2^p$, the M2 stops at w at the latest. (iii) Otherwise nobody set the counter to 0, so the process y at distance 2^p got counter 1 (F3). If $\max(y) > j$, it drops the M2; if not, it started to wait for j, is still waiting when the M2 comes (F2), and takes it. *Example:* in phase 1, M2(8) goes P7→P0→P1 (two links) and stops at P1, the process that received counter 1.

3.3.3. *In one phase each link carries at most one M1 and one M2.* For M1 this is (F1). The M2 of u stays between u and the next active process, and each active process creates at most one.

3.3.4–3.3.5 (the maximum survives). *Some process of every non-empty $A(p)$ holds the maximum M.* Let $u \in A(p)$ hold M and $|A(p)| \ge 2$. (1) Nobody has a larger max, so the counter of u's M1 is never reset to 0 early: it makes the next active process wait for M, or, if that one is farther than 2^p, the process at distance 2^p. (2) u receives the M1 of the previous active process, whose value is smaller (3.3.1), so u becomes passive and sends M2(M). (3) Nobody drops M2(M), and by (F2) it reaches the waiting process before any M1 of phase $p+1$; that process starts phase $p+1$ with M.

Lemma 3.3.6 (the timetable). *Let $v \in A(p)$ and $j = p$-max(v). Then (a) $d(\text{origin}(j), v) = 2^p - 1$; (b) for $p \ge 1$, every process y from* origin(j) *to v (inclusive) has* $\max(y) = j$ *right after it has processed its M1 of phase $p-1$ (the paper's p-max(y)); afterwards only M2s reach y before an M1 of phase p, so* $\max(y)$ *is still j when such an M1 arrives; (c) $d(w, v) \ge 2^p$ for every other $w \in A(p)$.*
Example. The value 8 of P6 is held by P6, P7 and P1 at the start of phases $0, 1, 2$: distances $0, 1, 3$ from its owner, so it moves by 1 and then by 2. After processing their M1 of phase 1, P6, P7, P0 and P1 all have max = 8.

Proof sketch (induction on p). For $p = 0$ all three parts are trivial. For the step, v entered $A(p)$ by taking an M2 with value j; let $u \in A(p-1)$ be the process that sent it, so $j = p$-max(u).

1. *u is the nearest process of $A(p-1)$ to the left of v,* because an M2 never passes the next active process.
2. $d(u, v) \le 2^{p-1}$ by 3.3.2.
3. $d(u, v) \ge 2^{p-1}$. Consider how v came to wait for j. If v was active in phase $p-1$, the bound is (c) for $p-1$. Otherwise v was passive (an old wait is cancelled by the first M1 of phase $p-1$

that passes it); the M1 of phase $p-1$ it received came from u (step 1), and a passive process starts to wait only on counter 1, that is, exactly 2^{p-1} from u (F3).

So *in phase $p-1$ the value moves exactly 2^{p-1} links.* From this:

(a) By induction $d(\text{origin}(j), u) = 2^{p-1} - 1$, and $(2^{p-1} - 1) + 2^{p-1} = 2^p - 1$.

(b) Split the segment at u. *After u up to v:* these processes received u's M1 of phase $p-1$ and later forwarded or took M2(j), so their max was at most j. Nobody here set the counter to 0 (that process would have dropped the M2), and it cannot have run down, since $d(u, v) = 2^{p-1}$. So they all stored j (B3 with $c \geq 1$, or B1 for an active v). *From* origin(j) *to u:* by (b) for $p-1$ these processes had j after their M1 of phase $p-2$ (for $p = 1$ the segment is just u), and only M2s reached them before their M1 of phase $p-1$. That M1 comes from the previous process x of $A(p-1)$, which lies to the left of origin(j), because $d(x, u) \geq 2^{p-1} > d(\text{origin}(j), u)$ by (c) for $p-1$. Its value is smaller than j (otherwise u would not have sent an M2), so their max stays j.

(c) The segment from origin(j) to v has 2^p processes. Another $w \in A(p)$ with $d(w, v) < 2^p$ would lie in it and hold j by (b), since taking an M2 does not change max, contradicting 3.3.1. □

Corollary 3.3.7 (few active processes). By 3.3.6(c), neighbouring active processes are at least 2^p apart, and the gaps around the ring add up to n, so $|A(p)| \cdot 2^p \leq n$. (If $|A(p)| = 1$ and $p \geq 1$: some process took an M2 of phase $p-1$, which needs $|A(p-1)| \geq 2$, because a lone active process receives no M1 and sends no M2; so $2^p \leq |A(p-1)|\, 2^{p-1} \leq n$.) For $p = \lfloor \log n \rfloor$ this leaves one active process, so there are at most $\lfloor \log n \rfloor + 1$ phases.

Why few processes are active, intuitively. A surviving value j "claims" the stretch from origin(j) to the active process carrying it, where everybody stores j as max (3.3.6(b)). Stretches of different values are disjoint, and their length doubles every phase. This does *not* mean that the number of active processes halves in every phase. On the ring $7, 3, 4, 5, 8, 2, 6, 1$ it goes $8 \to 3 \to 2 \to 1$. In phase 1 the value 7 survives although its active neighbour holds 8: P1 (value 7) beats 6, and its M2(7) is taken by the virtual candidate P3 at distance 2 before it can reach P5, which holds 8. The bound $n/2^p$ still holds ($8/4 = 2$), and that is all the analysis needs.

Termination and a unique leader.

- *A leader appears.* In the phase $P \leq \lfloor \log n \rfloor$ with a single active process v, this process holds M (3.3.5). No active process stops its M1, so the M1 reaches origin(M), which lies $2^P - 1$ links to the left of v by 3.3.6(a) and receives its own id.
- *No false leader.* Suppose the M1 with value j from $v \in A(p)$ reaches origin(j). Since origin(j) lies to the left of v (3.3.6(a)), the M1 passed every process outside the segment from origin(j) to v. Every other process of $A(p)$ lies outside this segment (3.3.6(c)) and would have stopped the M1. So $|A(p)| = 1$ and $j = M$.
- *No deadlock.* Every active process receives the M1 of the previous one (F1), and every M2 ends, so every phase finishes; by 3.3.5 the M2 with M is taken, so the next phase is not empty, and by 3.3.7 this stops after at most $\lfloor \log n \rfloor + 1$ phases.

The paper's Algorithm B has no explicit stop; the test "an arriving value equals *id*" is the optional change suggested at the end of its Section 3.2. We test only M1: an M2 with value j never reaches origin(j), because it would have to cross $n - 2^p + 1 \leq 2^p$ links, i.e. $n < 2^{p+1}$, while an M2 is sent only if $|A(p)| \geq 2$, i.e. $n \geq 2^{p+1}$ (3.3.7). Our implementation then sends one announcement around the ring (n more messages) so that everybody learns the result.

4 Message complexity

M1s. In every phase except the last, each link carries exactly one M1 (F1); the last phase has at most n. With at most $\log n + 1$ phases (3.3.7), $\#\text{M1} \leq n \log n + n$.

M2s: runs and silence. The *run* of an M2 is its creator together with the processes that forwarded it; its length is the number of links the M2 crossed, at most 2^p by 3.3.2. Runs start at their creators, which are in $A(p)$, and are disjoint (3.3.3).

Lemma 3.3.8 (silence after every run). *Let u create an M2 in phase p, let f be the last process of its run, and let g be the process with $d(f,g) = 2^p - 1$. Then no process strictly after f up to g sends an M2 in phase p.*
Proof sketch. The run of u ended at f, and a new run starts only at an active process that creates an M2. So it suffices to show that no such process lies after f up to g. Let w be the next process of $A(p)$ after u.

- p-max$(w) < p$-max(u). Then w waits and creates nothing. The next active process z after w has $d(u,z) = d(u,w) + d(w,z) \geq 2^p + 2^p$ by 3.3.6(c) (if $z = u$, read $d(u,z)$ as n). But $d(u,g) = d(u,f) + d(f,g) \leq (2^p - 1) + (2^p - 1) < 2^{p+1}$, because a run has at most 2^p senders. So z lies beyond g.
- p-max$(w) = k > p$-max(u). By 3.3.6(a,b) all processes from origin(k) to w hold k, and origin(k) lies after u, since $d(\text{origin}(k), w) = 2^p - 1 < 2^p \leq d(u,w)$. When u's M2 comes, origin(k) is passive (u's smaller value cannot make it a candidate) and its max $\geq k$, so it drops the M2 if nobody did before; f lies strictly before origin(k). Hence $d(f,w) \geq 1 + (2^p - 1) = 2^p > d(f,g)$: w, and every active process after it, lies beyond g. □

Example. In phase 1 of the running example the run of M2(8) is P7, P0, so $f =$ P0 and g is $2^1 - 1 = 1$ link further: $g =$ P1, which indeed sends no M2. In phase 0 the lemma is empty ($2^0 - 1 = 0$), and three of the eight processes send an M2.

Corollary 3.3.9 (about $n/2$ M2s per phase). Let $C_1, \ldots, C_r$ be the runs of phase p in ring order. After each run come at least $2^p - 1$ silent processes before the next one, so

$$n \;\geq\; \sum_{i=1}^{r} \big(|C_i| + 2^p - 1\big).$$

Since $|C_i| \leq 2^p$, we have $2^p - 1 \geq |C_i| \,\frac{2^p-1}{2^p}$, hence $|C_i| + 2^p - 1 \geq |C_i| \,\frac{2^{p+1}-1}{2^p}$. Every M2 of phase p is sent by one process of one run, so summing gives

$$\#\mathsf{M2}_p \;=\; \sum_i |C_i| \;\leq\; n\,\frac{2^p}{2^{p+1}-1} \;=\; \frac{n}{2} + \frac{n}{2(2^{p+1}-1)}.$$

The factor $2^p/(2^{p+1}-1)$ is $1, \frac{2}{3}, \frac{4}{7}, \frac{8}{15}, \ldots \to \frac{1}{2}$, so for larger p only about half of the processes send an M2.

Theorem 3.3.10 (fewer than $\frac{3}{2}n\log n + \frac{8}{3}n$ messages). In the last phase $P \leq \lfloor \log n \rfloor$ only one process is active and its M1 ends at the owner, so nobody sends an M2; M2s are sent only in phases $0, \ldots, P-1$, at most $\lfloor \log n \rfloor$ phases. Summing Corollary 3.3.9 over them,

$$\#\mathsf{M2} \;\leq\; \frac{n}{2}\log n \;+\; \frac{n}{2}\Big(1 + \tfrac{1}{3} + \tfrac{1}{7} + \tfrac{1}{15} + \cdots\Big) \;\approx\; \tfrac{1}{2}n\log n + 0.80\,n \;<\; \tfrac{1}{2}n\log n + \tfrac{5}{3}n.$$

The series is about 1.607; it is at most $1 + \frac{1}{2} + \frac{1}{4} + \cdots = 2$, because $\frac{1}{2^{p+1}-1} \leq \frac{1}{2^p}$. The paper's $\frac{5}{3}n$ is a generous bound. Adding $\#\mathsf{M1} \leq n\log n + n$ gives the total $< \frac{3}{2}n\log n + \frac{8}{3}n$. The paper also notes the price in *bits*: every M1 now carries a $\log n$-bit counter, so, measured in bits, B is worse than an optimised Algorithm A.

5 Implementation and measurements

The Go code is in the folder `code/` of the repository (link on the title page). It was written from scratch: the Algorithm B of the course's Go framework (`distributed-framework`) runs in the synchronous

model and always starts the counter at 1 instead of 2^p, so it loses change 3. The code follows the course guidelines for asynchronous algorithms:

- **one goroutine per process** (`node.run` in `network.go`); links are channels with buffer 1 (the minimum for $n = 1$, where a process sends to itself), and between steps a goroutine only stores arriving messages in its FIFO `inbox`;
- **the state is a separate type** (`State`, file `state.go`); **one message is handled by one pure function**: `handleB(State, Message)` returns the new state and the messages to send (`algorithm_b.go`), with step B3 in `passiveB` and the wake-up in `startB`; `node.step` loads the state, takes one message, calls the handler, sends and saves;
- **random order**: the scheduler in `Run` picks, with a seeded generator, a process that has not woken up yet or has a message waiting, and lets it take one step; before every step it waits for the global `Delay` (flag `-delay`);
- **logging**: every step prints the handled message and the new state, then lines like `P2 sent M2(value=6) to P3`; the summary gives the leader, phases, M1 and M2 counts and the bound;
- **tests** (`dkr_test.go`): the right leader for $n = 1, \dots, 70$ and four kinds of rings, the bound of Theorem 3.3.10, Lemma 3.3.6(a,c), Corollaries 3.3.7 and 3.3.9, and that the schedule does not change any count;
- **comparison**: Algorithm A (`algorithm_a.go`, flag `-algo A`).

`handleB` follows the table of Section 2 row by row: the stop test, `case Active` (B1), `case Waiting` (B2; the next M1 gets counter `1<<s.Phase`), and `passiveB` (B3). Two `panic` calls mark situations the paper rules out (an active process receiving M2, a waiting one receiving an M2 with another value); the tests never trigger them. `handleAnnouncement` spreads the result, and `record` counts messages per phase.

Scaling with n (`go run . -bench -seed 7`; full table in `report/bench.txt`). Counts are election messages M1 + M2, without the n announcement messages. *Random*: average over 20 random rings. *Bit-reversal*: ids in bit-reversed order of positions, e.g. $1, 5, 3, 7, 2, 6, 4, 8$ for $n = 8$; every other value is a local maximum in every phase, so exactly half survive and there are $\log n + 1$ phases. This is the worst case for the number of phases, though not quite for the number of messages: on the ring $7, 3, 4, 5, 8, 2, 6, 1$ of Section 3, B sends 38 instead of 37.

	random (average)				bit-reversal			
n	phases	B	$\frac{\text{B}}{n \log n}$	$\frac{\text{A}}{n \log n}$	phases	B	$\frac{\text{B}}{n \log n}$	$\frac{\text{A}}{n \log n}$
16	4.0	75	1.170	1.700	5	97	1.516	2.250
64	5.3	408	1.062	1.433	7	577	1.503	2.167
256	7.2	2204	1.076	1.400	9	3073	1.500	2.125
1024	9.1	10952	1.070	1.350	11	15361	1.500	2.100
4096	10.8	51867	1.055	1.375	13	73729	1.500	2.083

On bit-reversal rings B sends $\frac{3}{2} n \log n + 1$ messages (n M1s and $n/2$ M2s per phase, plus one M1 in the last phase) and A exactly $2n \log n + n$, so both leading constants are attained; only the $\frac{8}{3}n$ term of the theorem is loose. On random rings the number of phases grows like $\log n$ and both algorithms stay well below their worst case; on sorted rings (`-layout asc/desc`) two phases are enough.

What each change buys ($n = 1024$; *crowded*: ids $1, \dots, 1008$ in order followed by the 16 largest ids in bit-reversal order, so the active processes end up on a short arc; *random*: average over 30 rings).

	bit-reversal		crowded		random	
	M2	total	M2	total	M2	total
A	10240	21504	4096	9216	6315	13653
A + 1	5120	16384	2048	7168	3267	10606
A + 1 + 2	5120	16384	2048	7168	2790	10129
B = A + 1 + 2 + 3	5120	15361	1040	6145	2002	11057

Change 1 halves the number of M2s everywhere, change 2 helps when values are mixed, and change 3 halves it again on the crowded ring. The totals also reflect the stopping rule: the A rows stop when the value comes back to its sender, B at the owner; with B's rule, A+1+2 would also send 15361 and 6145 in the first two columns. On random rings the counter adds phases (9 instead of 7 on average), because survivors need not be local maxima; B sends about 1700 more M1s and 800 fewer M2s, 9% more in total than A+1+2 in this sample (2–9% in others). The counter buys the worst-case guarantee, which is what the paper optimises. The A+1 and A+1+2 rows come from a separate simulation, not part of the submitted code; the random column is a different sample from the table above.

6 Points that are easy to misunderstand

- **Only M2 is limited, not M1.** An M1 always travels to the next active process, however far: that process needs it to compare. The counter only decides who becomes a candidate on the way; after 0 the M1 is only relayed.
- **Waiting is not yet active.** A virtual candidate becomes waiting when the M1 with counter 1 passes it, and active only later, when the M2 arrives. An active process stops being active as soon as it receives an M1: a bigger value makes it wait, a smaller one makes it passive and send an M2.
- **Why a waiting process never forwards M2.** The M2 carries the value the process is waiting for (the paper notes $j = \max(v)$ in step B2), so there is nothing to forward; stopping there is what bounds the length of an M2 by 2^p.
- **Who drops an M2.** Any passive process whose max is larger, including the next active process itself after it won its comparison and became passive (in the running example $5, 2, 6, 3, 1, 4, 8, 7$, P6 drops M2(4) in phase 0 because it holds 8).
- **Why $\geq$ (not $>$) in B3.** The next active process after a virtual candidate may already hold the value: it adopted it in B1 from an M1 with counter 0. In a later phase that value's M1 must count down there, not be reset to 0; otherwise the M2 could travel more than 2^p links.

7 How to run the code

Install Go 1.21 or newer (`https://go.dev/dl/`, or `brew install go` on macOS). The code has no other dependencies. Every run prints its seed, and `-seed` reproduces it; all flags are listed by `go run . -h` and in `README.md`.

```
git clone https://github.com/RomanVolochai/distributed-systems-unidir-leader-finder-b.git
cd distributed-systems-unidir-leader-finder-b/code
go run .                                        # random ring of 8 processes, full log
go run . -ids 5,2,6,3,1,4,8,7                   # the ring from the talk (26 + 8 announcement)
go run . -file examples/ring8.txt -delay 300ms # follow it live
go run . -n 1000 -quiet -seed 42                # summary only (-layout bitrev: most phases)
go run . -n 1000 -quiet -seed 42 -algo A        # the same ring, Algorithm A
go run . -bench -seed 7                         # the scaling table above
go test                                         # all checks, prints "ok"
```